

Program No:14

Aim

Develop a concurrent file server that will provide the file requested by the client if it exists. If not, the server sends an appropriate message to the client. The server should also send its process ID (PID) to clients for displaying along with the file contents or with the message.

Server Algorithm

1. **Start** the server.
2. Set the **host name** and **port number** inside the program.
3. Create a **TCP socket**.
4. Bind the socket to the specified host and port.
5. Make the server listen for incoming client connections.
6. Accept a client connection.
7. Create a **new thread** to handle the client.
8. Receive the **file name** from the client.
9. Get the **server process ID (PID)**.
10. Check whether the requested file exists.
11. If the file exists:
 - Open the file.
 - Read its contents.
 - Send the **PID and file contents** to the client.
12. If the file does not exist:
 - Send the **PID and "File not found"** message to the client.
13. Close the client connection.
14. Return to Step 6 to accept another client.
15. **Stop**.

Client Algorithm

1. **Start** the client.
2. Set the **server host name and port number**.
3. Create a **TCP socket**.
4. Connect to the server.
5. Ask the user to enter the **file name**.
6. Send the file name to the server.
7. Receive the response from the server.
8. Display the **server PID and file contents/message**.
9. Close the socket.
10. **Stop**.

Server Program

```
import socket
import threading
import os
HOST = "localhost"
PORT = 5000
def handle_client(client):
    filename = client.recv(1024).decode()
```

```

pid = os.getpid()
if os.path.exists(filename):
    with open(filename, "r") as f:
        data = f.read()
        message = f"PID: {pid}\n{data}"
else:
    message = f"PID: {pid}\nFile not found"
client.send(message.encode())
client.close()

server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind((HOST, PORT))
server.listen(5)
print("Server started")
print("Host:", HOST)
print("Port:", PORT)
print("PID:", os.getpid())
while True:
    client, address = server.accept()
    thread = threading.Thread(target=handle_client, args=(client,))
    thread.start()

```

Client

```

import socket
HOST = "localhost"
PORT = 5000
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect((HOST, PORT))
filename = input("Enter file name: ")
client.send(filename.encode())
data = client.recv(4096).decode()
print("\nServer Response:")
print(data)
client.close()

```

Result

The concurrent file server was successfully implemented. It sends the requested file contents along with the server PID, or a "File not found" message if the file does not exist.